L'objectif de ce TP est de se familiariser avec certaines **opérations de base en Text mining (fouille de textes)**.

1 Opérations sur des textes bruts

Peut-être encore davantage que d'autres tâches de fouille de données, la fouille de textes et la recherche de documents posent le problème du prétraitement. Les textes et documents sont en effet généralement disponibles sous des formes et des formats divers : en html, xml, doc, pdf, etc. Dans le cadre limité de ce tp, nous supposerons que les documents ont été traités pour être disponibles sous la forme de corpus exploitables, et nous allons nous focaliser sur des opérations d'analyse globale et de ré-description.

Commençons par invoquer nltk et les librairies utiles :

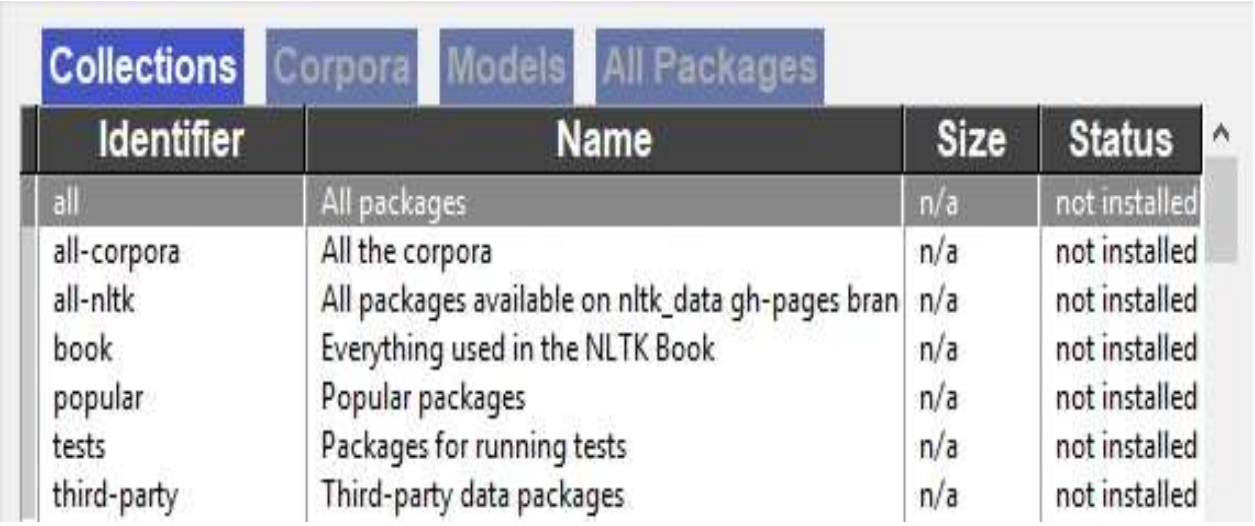
```
In [1]: import nltk
```

Il peut-être utile de lancer la commande : nltk.download() pour charger les ouvrages. Nous chargeons ensuite un texte sur lequel nous ferons diverses opérations.

```
In [2]: from nltk.book import *
```

Cette commande charge à la fois 9 textes « classiques » et les fonctions prédéfinies de nltk.

Le text2 est celui du roman *Sense and sensibility* de Jane Austen, écrit en 1811.



Collections | Corpora | Models | All Packages

Identifier	Name	Size	Status
all	All packages	n/a	not installed
all-corpora	All the corpora	n/a	not installed
all-nltk	All packages available on nltk_data gh-pages bran	n/a	not installed
book	Everything used in the NLTK Book	n/a	not installed
popular	Popular packages	n/a	not installed
tests	Packages for running tests	n/a	not installed
third-party	Third-party data packages	n/a	not installed

1.1 Comptages et fréquences

Nous commençons par quelques **opérations élémentaires** : calcul de la *longueur du texte* en mots, *taille du vocabulaire* (nombre de mots différents), *diversité du vocabulaire* (rapport de la longueur du texte sur la taille du vocabulaire).

Calcul du **nombre de mots** de *Sense and Sensibility* :

```
In [3]: len(text2)
```

Calcul de la **taille du vocabulaire** utilisé (pour cela on calcule d'abord l'ensemble des mots par la fonction set):

```
In [4]: len(set(text2))
```

Nous allons maintenant définir une fonction de calcul de la **diversité lexicale**.

```
def lexical_diversity(text):
return len(text) / len(set(text))
```

Appliquez la fonction à *Sense and Sensibility*. Qu'obtenez-vous ?

Appliquez cette même fonction au text1 (*Moby Dick* de Herman Melville en 1851) et au text6 (*Monty Python and the Holy Grail*, film britannique sorti en 1975 et écrit et réalisé par Terry Gilliam et Terry Jones des Monty Python). Faut-il en conclure quelque chose ?

Il est également possible de compter le nombre de fois où un mot apparaît dans un texte. Par exemple (ici on applique une méthode 'count' à un objet, le texte) :

```
In [13]: text6.count("love")
Out[13]: 0
```

1.2 Comptage de mots dans différents documents

Nous allons comparer des documents en fonction de la fréquence de mots dans chacun d'eux.

On charge d'abord le corpus Brown qui contient des extraits de journaux et revues, et on met dans news_text la liste des mots rencontrés. On va alors compter le nombre d'occurrences de mots modaux.

```
In [14]: from nltk.corpus import brown
In [15]: news_text = brown.words(categories='news')
In [16]: fdist = nltk.FreqDist([w.lower() for w in news_text])
In [17]: modals = ['can', 'could', 'may', 'might', 'must', 'will']
In [18]: for m in modals:
                print(m + ':', fdist[m])
can: 94
could: 87
may: 93
might: 38
must: 53
will: 389
```

Faites la même chose avec la catégorie 'science_fiction' (au lieu de 'news') et avec les mots commençant par wh (what, where, who, why, …).

Nous allons maintenant faire une comparaison systématique entre catégories sur les mots modaux.

```
In [19]: cfd = nltk.ConditionalFreqDist(
.....: (genre, word)
.....: for genre in brown.categories()
.....: for word in brown.words(categories = genre))
In [20]:
In [21]: genres = ['news', 'religion', 'hobbies', 'science_fiction', 'romance', 'humor']
In [22]: cfd.tabulate(conditions = genres, samples = modals)
```

	can	could	may	might	must	will
News	93	86	66	38	50	389
Religion	82	59	78	12	54	71
Hobbies	268	58	131	22	83	264
science_fiction	16	49	4	12	8	16
Romance	74	193	11	51	45	43
Humor	16	30	8	8	9	13

Quel est le mot modal le plus fréquent dans la catégorie ‘news’, et dans la catégorie ‘romance’ ? Pouvait-on s’y attendre ?
Pourrait-on distinguer les différentes catégories en utilisant la fréquence des mots ?

1.3 Les stopwords

On va d’abord examiner quels sont les mots les moins fréquents dans *Sense and Sensibility*. Pour cela, on calcule les mots présents dans ce roman et on en **retire tous les mots usuels en anglais**. Ceux-ci sont obtenus grâce à un corpus spécial de nltk. Voici donc la fonction qui fait ce travail :

```
def unusual_words(text):
text_vocab = set(w.lower() for w in text if w.isalpha())
# on ne considere que les mots et on enleve les majuscules
english_vocab = set(w.lower() for w in nltk.corpus.words.words())
unusual = text_vocab.difference(english_vocab)
return sorted(unusual)
```

Ce qui donne :

```
>>> unusual_words(nltk.corpus.gutenberg.words('austen-sense.txt'))
```

['abbeyland', 'abhorrence', 'abominably', 'abridgement', 'accordant', 'accustomary', 'adieus', 'affability', 'affectedly', 'aggrandizement', 'alighted', 'allenham','amiably', 'annamaria', 'annuities', 'apologising', 'arbour', 'archness', ...]

Essayez sur le corpus de ‘chat’ :

```
>>> unusual_words(nltk.corpus.nps_chat.words())
```

Il existe un **corpus de stopwords** disponible dans nltk. Il comporte 127 mots.

```
>>> from nltk.corpus import stopwords
>>> stopwords.words('english')
```

['a', "a's", 'able', 'about', 'above', 'according', 'accordingly', 'across', 'actually', 'after', 'afterwards', 'again', 'against', "ain't", 'all', 'allow', 'allows', 'almost', 'alone', 'along', 'already', 'also', 'although', 'always', ...]

Nous allons définir une fonction qui calcule la fraction de mots d'un texte qui ne font pas partie des stopwords. Nous l'appliquons alors au corpus des dépêches Reuters.

```
>>> def content_fraction(text):
stopwords = nltk.corpus.stopwords.words('english')
...  content = [w for w in text if w.lower() not in stopwords]
...  return len(content) / len(text)
...
>>> content_fraction(nltk.corpus.reuters.words())
```

0.6599769539328526

2 Sélection de descripteurs pertinents

2.1 Utilisation de WordNet

Nous allons maintenant utiliser le réseau sémantique WordNet pour étudier les relations entre mots.

```
>>> from nltk.corpus import wordnet as wn
>>> wn.synsets('motorcar')
[Synset('car.n.01')
```

Le mot 'motorcar' n'a qu'un sens dans WordNet comme le montre le fait que l'ensemble des synonymes a un seul élément. Qu'en est-il avec le mot 'dish' ?

On peut examiner les différents noms correspondant à l'unique sens de 'motorcar' :

```
>>> wn.synset('car.n.01').lemma_names()
['car', 'auto', 'automobile', 'machine', 'motorcar']
```

Combien y a-t-il de mots dans WordNet correspondant au premier synonyme de 'dish', et combien pour le premier sens en tant que verbe ?

On peut obtenir la définition d'un sens du mot. De même, on peut demander un exemple d'usage:

```
>>> wn.synset('car.n.01').definition()
'a motor vehicle with four wheels; usually propelled by an internal combustion engine'
>>> wn.synset('car.n.01').examples()
['he needs a car to get to work']
```

Quelle est la définition du 3 ème sens de 'dish' en tant que nom, et quel exemple est fourni ? Nous allons maintenant nous intéresser aux sous-classes d'un mot (ses hyponymes) et à ses surclasses (ses hypernymes).

Sélectionnons le 1 er sens du mot 'motorcar', puis cherchons ses hyponymes dans WordNet, avant d'en donner les mots (lemmas) qui les décrivent :

```
>>> motorcar = wn.synset('car.n.01')
>>> types_of_motorcar = motorcar.hyponyms()
>>> types_of_motorcar[26]
Synset('ambulance.n.01')
```

```
>>> sorted([lemma.name for synset in types_of_motorcar for lemma in synset.lemmas])
['Model_T', 'S.U.V.', 'SUV', 'Stanley_Steamer', 'ambulance', 'beach_waggon','beach_wagon', 'bus', 'cab',
'compact', 'compact_car', 'convertible','coupe', 'cruiser', 'electric', 'electric_automobile',
'electric_car','estate_car', 'gas_guzzler', 'hack', 'hardtop', 'hatchback', 'heap','horseless_carriage', 'hot-rod',
'hot_rod', 'jalopy', 'jeep', 'landrover','limo', 'limousine', 'loaner', 'minicar', 'minivan', 'pace_car',
'patrol_car','phaeton', 'police_car', 'police_cruiser', 'prowl_car', 'race_car', 'racer','racing_car', 'roadster',
'runabout', 'saloon', 'secondhand_car', 'sedan','sport_car', 'sport_utility', 'sport_utility_vehicle',
'sports_car', 'squad_car', 'station_waggon', 'station_wagon', 'stock_car', 'subcompact',
'subcompact_car','taxi', 'taxicab', 'tourer', 'touring_car', 'two-seater', 'used-car', 'waggon','wagon']
```

Faites de même avec ‘dish’.

Il est facile d’obtenir de la même manière les hypernymes :

```
>>> motorcar.hypernyms()
[Synset('motor_vehicle.n.01')]
```

Il est intéressant d’examiner l’ensemble des chemins ascendants vers la racine de l’ontologie WordNet. Dans le cas de ‘motorcar’, il existe deux chemins distincts :

```
>>> paths = motorcar.hypernym_paths()
>>> len(paths)
2
>>> [synset.name for synset in paths[0]]
['entity.n.01', 'physical_entity.n.01', 'object.n.01', 'whole.n.02', 'artifact.n.01', 'instrumentality.n.03',
'container.n.01', 'wheeled_vehicle.n.01','self-propelled_vehicle.n.01', 'motor_vehicle.n.01', 'car.n.01']
>>> [synset.name for synset in paths[1]]
['entity.n.01', 'physical_entity.n.01', 'object.n.01', 'whole.n.02', 'artifact.n.01', 'instrumentality.n.03',
'conveyance.n.03', 'vehicle.n.01', 'wheeled_vehicle.n.01','self-propelled_vehicle.n.01',
'motor_vehicle.n.01', 'car.n.01']
```

Qu’en est-il de ‘dish’ ?

L’existence de WordNet permet de calculer des distances sémantiques entre mots. Par exemple, on peut considérer que deux mots sont d’autant plus proches sémantiquement qu’ils partagent un hyponyme à distance faible au-dessus d’eux.

Cherchons par exemple la similarité entre les mots ‘right_whale’, ‘orca’, ‘minke’, ‘tortoise’ et ‘novel’. On commence par charger le premier sens de chacun de ces mots :

```
>>> right = wn.synset('right_whale.n.01')
>>> orca = wn.synset('orca.n.01')
>>> minke = wn.synset('minke_whale.n.01')
>>> tortoise = wn.synset('tortoise.n.01')
>>> novel = wn.synset('novel.n.01')
```

On peut ensuite chercher l'hypernyme commun le plus bas dans l'ontologie :

```
>>> right.lowest_common_hypernyms(minke)
[Synset('baleen_whale.n.01')]
>>> right.lowest_common_hypernyms(orca)
[Synset('whale.n.02')]
>>> right.lowest_common_hypernyms(tortoise)
[Synset('vertebrate.n.01')]
>>> right.lowest_common_hypernyms(novel)
[Synset('entity.n.01')]
```

Il est clair que 'right_whale' et 'whale' sont proches, ce qui n'est pas le cas de 'right_whale' et de 'novel' dont l'hypernyme commun le plus spécifique est 'entity' qui est la racine de WordNet. On peut quantifier cette proximité en regardant la profondeur du premier hypernyme commun.

```
>>> wn.synset('baleen_whale.n.01').min_depth()
14
>>> wn.synset('whale.n.02').min_depth()
13
>>> wn.synset('vertebrate.n.01').min_depth()
8
>>> wn.synset('entity.n.01').min_depth()
0
```

Une mesure découlant de cette « distance arborée » est calculée par la fonction path_similarity. Elle retourne un nombre entre 0 et 1, 0 indiquant une similarité nulle alors que 1 est la similarité entre un terme et lui-même (la fonction retourne -1 si aucun chemin n'est trouvé entre les deux termes (e.g. entre un nom et un verbe).

```
>>> right.path_similarity(minke)
0.25
>>> right.path_similarity(orca)
0.16666666666666666
>>> right.path_similarity(tortoise)
0.076923076923076927
>>> right.path_similarity(novel)
0.04347826086956521
On peut aussi calculer la similarité (ici entre orque et vison) par :
>>> print "psim(orca,minke)=", orca.path_similarity(minke)
psim(orca,minke)= 0.142857142857
```

Quel degré de similarité trouvez-vous entre 'novel', 'romance', 'news' et 'report' ?

2.2 Stemming et lemmatisation

Souvent il est intéressant de ramener un texte à des mots racines afin d'éliminer les différences dues à des formes particulières des mots. Un procédé pour ce faire est le **stemming**. Il n'existe pas de manière unique de procéder et plusieurs « stemmers » sont donc disponibles.

Partons par exemple du texte suivant :

```
>>> raw = """DENNIS: Listen, strange women lying in ponds distributing swords
is no basis for a system of government. Supreme executive power derives from
... a mandate from the masses, not from some farcical aquatic ceremony."""
>>> tokens = nltk.word_tokenize(raw)
```

L'utilisation du stemmer de Porter, l'un des plus utilisés, conduira à :

```
>>> porter = nltk.PorterStemmer()
>>> [porter.stem(t) for t in tokens]
['DENNI', ':', 'Listen', ',', 'strang', 'women', 'lie', 'in', 'pond', 'distribut', 'sword', 'is', 'no', 'basi', 'for', 'a',
'system', 'of', 'govern', '.', 'Suprem', 'execut', 'power', 'deriv', 'from', 'a', 'mandat', 'from', 'the', 'mass', ',',
'not', 'from', 'some', 'farcic', 'aquat', 'ceremoni', '.']
```

Mais le stemmer de Lancaster produit :

```
>>> lancaster = nltk.LancasterStemmer()
>>> [lancaster.stem(t) for t in tokens]
['den', ':', 'list', ',', 'strange', 'wom', 'lying', 'in', 'pond', 'distribut', 'sword', 'is', 'no', 'bas', 'for', 'a', 'system',
'of', 'govern', '.', 'suprem', 'execut', 'pow', 'der', 'from', 'a', 'mand', 'from', 'the', 'mass', ',', 'not', 'from', 'som',
'farc', 'aqu', 'ceremony', '.']
```

La **lemmatisation** ne supprime les suffixes que si le mot résultant fait partie du dictionnaire. Par exemple :

```
>>> wnl = nltk.WordNetLemmatizer()
>>> [wnl.lemmatize(t) for t in tokens]
['DENNIS', ':', 'Listen', ',', 'strange', 'woman', 'lying', 'in', 'pond', 'distributing', 'sword', 'is', 'no', 'basis',
'for', 'a', 'system', 'of', 'government', '.', 'Supreme', 'executive', 'power', 'derives', 'from', 'a', 'mandate',
'from', 'the', 'mass', ',', 'not', 'from', 'some', 'farcical', 'aquatic', 'ceremony', '.']
```

2.3 Tagging

Afin de distinguer entre les différents sens possibles d'un mot, il est intéressant de chercher leur rôle grammatical dans le texte étudié. C'est en particulier la tâche du Part-Of-Speech tagging.

```
>>> text = nltk.word_tokenize("And now for something completely different")
>>> nltk.pos_tag(text)
[('And', 'CC'), ('now', 'RB'), ('for', 'IN'), ('something', 'NN'),
('completely', 'RB'), ('different', 'JJ')]
```

Le mot '*and'* est donc reconnu comme CC, c'est-à-dire une conjonction. Les mots '*now'* et '*completely'* comme des RB ou adverbes, '*for'* est IN, c'est-à-dire une préposition, '*something'* est NN, soit un nom et '*different'* est JJ, c'est-à-dire un adjectif.

Considérons un cas plus difficile incluant des homonymes :

```
>>> text = nltk.word_tokenize("They refuse to permit us to obtain the refuse permit")
```

On voit par exemple que le mot '*permit'* est bien reconnu comme un verbe, VB, et comme un nom, NN, de manière appropriée dans le texte. Il en est de même pour '*refuse'*.

2.4 Sélection de mots pertinents par tf-idf

L'analyse tf-idf permet de caractériser la pertinence de mots pour distinguer des textes pris dans un corpus. Un mot est ainsi d'autant plus pertinent pour distinguer un texte au sein d'un corpus que sa fréquence dans ce texte est différente de la fréquence moyenne dans le corpus.

Les fonctions tf, idf et tfidf sont définis dans nltk. Nous allons considérer trois textes du corpus book de nltk.

```
In [46]: from nltk.book import *
In [47]: text1
Out[47]: <Text: Moby Dick by Herman Melville 1851>
In [66]: mytexts = nltk.TextCollection([text1, text2, text3])
```

Calculons maintenant la fréquence du mot 'is' dans les trois textes :

```
In [77]: mytexts.tf('is',text1)
Out[77]: 0.0064987596762505796
In [78]: mytexts.tf('is',text2)
Out[78]: 0.0051421144826806807
In [79]: mytexts.tf('is',text3)
Out[79]: 0.0059646144223036365
```

Calculons l'*inverse document frequency* du mot 'companionableness' dans nos trois textes :

```
In [85]: mytexts.idf('companionableness')
Out[85]: 1.0986122886681098
```

Et **le score tf-idf** de ce mot :

```
In [86]: mytexts.tf_idf('companionableness', text2)
Out[86]: 7.7598765939715046e-06
```

On va maintenant chercher les mots les plus pertinents pour caractériser un texte. Pour cela on va ordonner les mots en fonction de leur score tf-idf pour ce document (et ce corpus).